

ft_calc

Your very first own calculator

Summary:
In this exercise, you will build a small command-line calculator, able to compute and to compare two integers.

Version: 1.0

Contents

I Introduction **2**

II Common Instructions **3**

III AI Instructions **4**

IV Mandatory part **5**

IV.1 Technical considerations 5

IV.2 Arguments. 5

IV.3 Operations. 6

IV.4 Errors 7

IV.5 Examples. 7

V Bonus part **8**

VI Submission and peer-evaluation **10**

Chapter I

Introduction

Rouen, 1642. Blaise Pascal is not even twenty years old. His father, in charge of collecting taxes in Normandy, spends his nights adding up endless columns of numbers, and Blaise spends his nights helping him.

Tired of it, he decides to build a machine that will do the job for them: a box full of gears and wheels, able to add and subtract two numbers. The *Pascaline* is one of the very first mechanical calculators in history.

Nearly four centuries later, your terminal can do almost anything... except help you with your taxes. Time to follow in Pascal's footsteps: you are going to build your own calculator.

And since we have made some progress since 1642, yours will not only add and subtract: it will also multiply, divide, and compare numbers.



```
Fun fact: in 1970, a programming language was named in his honour:
Pascal. Unlike his calculator, it could do a lot more than additions.
```

Chapter II

Common Instructions

- This exercise must be written in C.
- Your code must be written in accordance with the Norm. It will be checked with `norminette`, and you will receive a 0 if there is a Norm error.
- Your program must compile with `cc` and the flags `-Wall`, `-Wextra` and `-Werror`. If it does not compile, you will get 0.
- Your program must not quit unexpectedly (segmentation fault, bus error, double free, etc.). If this happens, your exercise will be considered non-functional and will receive a 0.
- Declaring global variables is strictly forbidden.
- You may only use the external functions listed in this subject. Using a forbidden function is considered cheating. Cheaters get -42, and this grade is non-negotiable.
- All your output must be done with `write`: `printf`, `fprintf`, `putchar` and the like are forbidden in every exercise.
- No Makefile is required: your program will be compiled with the command given in Chapter IV.
- We encourage you to test your program with many different inputs, including edge cases (negative numbers, zero, invalid signs, missing arguments...). Your evaluators will do it, for sure.
- Submit your work to the assigned Git repository. Only the work in the Git repository will be graded.

Chapter III

AI Instructions

Context

This exercise is designed to help you consolidate the fundamental building blocks of your 42 training. To properly anchor these skills, you need to struggle with them yourself: true learning requires genuine intellectual effort, through challenge, repetition and peer-learning exchanges.

Main message

- ☛ Build strong foundations without shortcuts.
- ☛ The learning journey is more important than the result.
- ☛ An exercise solved by an AI is an exercise you did not solve.

Learner rules

- You should apply reasoning to your exercise, especially before turning to AI.
- You should not ask for direct answers to the AI.
- You must be able to explain every line you submit.

✓ Good practice:

> My program says that 6 * 7 is an error. A peer suggests I type echo 6 * 7 in my terminal. I discover what my shell does with the * character. I get it.

✗ Bad practice:

> I paste the subject into an AI and copy its answer. During the evaluation, I can't explain why my program checks the arguments in this order. I fail. During the exam (no AI) I'm stuck again. I fail.

Chapter IV

Mandatory part

Turn-in directory	`ex01/`
Files to Submit	`ft_calc.c, and any other .c or .h file you need`
Compilation	`cc -Wall -Wextra -Werror *.c`
Arguments	`<int> <sign> <int>`
External Function	`write`
Description	`A calculator that computes or compares two integers.`

IV.1 Technical considerations

✓ **Allowed:**

- The `write` function.
- Helper functions (your own `ft_putnbr`, `ft_atoi`...), spread over as many .c files as you need.

✗ **Forbidden:**

- Any other external function (`printf`, `atoi`, `strcmp`...).
- Global variables.



`The Norm allows at most 5 functions per file, and a calculator needs a few more. Split your code into several .c files, and put your prototypes in a header.`

IV.2 Arguments

Your program takes exactly three arguments: a number, a sign, and another number:

```
./a.out <int> <sign> <int>
```

A **valid number** is made of an optional sign (`+` or `-`), followed by at least one digit, and nothing else:

- valid numbers: `42`, `-42`, `+42`, `0`, `007`;
- invalid numbers: `4a`, `4.2`, `-`, `+-4`, `" 42"`, `""` (empty string).



You don't have to handle overflows in the mandatory part: during its evaluation, the numbers given and the results of the operations will always fit in an int. Handling them is the bonus part of this exercise.

IV.3 Operations

A **valid sign** is one of the following, and nothing else:

Sign	Operation	Your program displays...
`+`	addition	the result
`-`	subtraction	the result
`*`	multiplication	the result
`/`	integer division	the result
`==`	is equal to	`true` or `false`
`!=`	is different from	`true` or `false`
`<`	is lower than	`true` or `false`
`>`	is greater than	`true` or `false`

- The result is displayed on the standard output, followed by a newline, and your program returns 0.
- The division is an integer division, exactly like the `/` operator in C: `85 / 2` gives `42`, and `-85 / 2` gives `-42`.



Your shell gives a special meaning to some of these characters: `*` is replaced by the names of the files of the current directory, `<` and `>` are redirections, and zsh even refuses `==`.

When you test your program, protect the sign with single quotes: `./a.out 6 '*' 7`

IV.4 Errors

The arguments are checked from left to right: only the **first** error found is displayed. Every error message is displayed on the **error output**, followed by a newline, and your program returns 1.

Situation	Message
wrong number of arguments	`Error: usage ./a.out <int> <sign> <int>.`
invalid 1st argument	`Error: arg 1 must be a number.`
invalid 2nd argument	`Error: arg 2 must be a sign (+ - * / == != < >).`
invalid 3rd argument	`Error: arg 3 must be a number.`
division by zero	`Error: division by zero.`

IV.5 Examples

```
$> ./a.out 21 + 21 | cat -e
42$
$> ./a.out 10 - 52 | cat -e
-42$
$> ./a.out 6 '*' 7 | cat -e
42$
$> ./a.out -85 / 2 | cat -e
-42$
$> ./a.out +7 + -7 | cat -e
0$
$> ./a.out 42 '==' 42 | cat -e
true$
$> ./a.out 42 '!=' 42 | cat -e
false$
$> ./a.out -42 '<' 0 | cat -e
true$
$> ./a.out 1 '>' 2 | cat -e
false$
$>
```

Errors are displayed on the error output, so they are not affected by `| cat -e`:

```
$> ./a.out 42 / 0
Error: division by zero.
$> ./a.out 4a + 2
Error: arg 1 must be a number.
$> ./a.out 4 x 2
Error: arg 2 must be a sign (+ - * / == != < >).
$> ./a.out 4 + -
Error: arg 3 must be a number.
$> ./a.out 4 2
Error: usage ./a.out <int> <sign> <int>.
$> echo $?
1
$>
```

Chapter V

Bonus part

Like the wheels of the Pascaline, which went back to zero once they had reached their maximum, an `int` has its limits: it can only hold values from -2147483648 to 2147483647. Beyond that, C does not warn you: the result is simply wrong, and for signed integers, it is even undefined behavior.

For the bonus, your calculator must not be fooled anymore. It handles three new errors:

Situation	**Message**
the 1st argument does not fit in an int	`Error: arg 1 must fit in an int.`
the 3rd argument does not fit in an int	`Error: arg 3 must fit in an int.`
the result does not fit in an int	`Error: result does not fit in an int.`

- These errors follow the same rules as the others: they are displayed on the error output, followed by a newline, your program returns 1, and the arguments are still checked from left to right.
- Everything else stays exactly as described in the mandatory part. The bonus is written in the same files: no extra file is needed.



`Once an overflow has happened, it is too late to detect it. Check before computing, or compute in a larger type such as long.`

The bonus part will only be assessed if the mandatory part is **PERFECT**. Perfect means the mandatory part has been integrally done and works without malfunctioning. If you have not passed **ALL** the mandatory requirements, your bonus part will not be evaluated at all.

Examples:

```
$> ./a.out 2147483647 + 0 | cat -e
2147483647$
$> ./a.out 000000000000000000042 + 0 | cat -e
42$
$> ./a.out 2147483647 + 1
Error: result does not fit in an int.
$> ./a.out -2147483648 / -1
Error: result does not fit in an int.
$> ./a.out 65536 '*' 32768
Error: result does not fit in an int.
$> ./a.out 2147483648 '==' 2147483648
Error: arg 1 must fit in an int.
$> ./a.out 42 '*' 99999999999999999
Error: arg 3 must fit in an int.
$>
```

Chapter VI

Submission and peer-evaluation

Submit your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the peer evaluation. Make sure to double-check the names of your files and folders to ensure they are correct.



`Your files must be placed in a folder named ex01 at the root of your repository: ex01/ft_calc.c, plus your other .c and .h files.`

During the evaluation, a brief **modification of the exercise** may be requested. This could involve a minor behaviour change, a few lines of code to write or rewrite, or an easy-to-add feature.

This step is meant to verify your actual understanding of the exercise. It should be feasible within a few minutes, in any development environment you choose.

For this exercise, your evaluator can for example ask you to support a new sign, such as `%` (modulo), `<=` or `>=`.